

Verus LLC

Coding Exercise – Company API and Frontend Implementation

Objective

Design and implement a Web API and a simple frontend application that demonstrate real-world software engineering practices. The solution should showcase sound API design, validation, error handling, clean architecture, and frontend–backend integration.

1. Company Submission Capability

Implement HTTP-based functionality that allows clients to submit company information.

Requirements

- Accept a JSON payload representing a Company
 - Validate all incoming data according to the rules defined in Section 2
 - Persist valid company records in an in-memory data store
 - Return appropriate HTTP responses:
 - **201 Created** or **200 OK** on success
 - **400 Bad Request** on validation failure, with a clear and concise error message
-

2. Data Validation Rules

The API must enforce the following validation rules:

Company Name

- Must not be empty
- Must contain at least a few characters

Website URL

- Must be a valid, well-formed URL
- Example: <https://www.example.com>

Relevance Validation

- The Company Name must be relevant to the Website URL
- Define and implement a relevance strategy using modern, reasonable, and explainable techniques (e.g., domain matching, content inspection, or equivalent logic)

Error Handling

- Any validation failure must:
 - Return **HTTP 400 Bad Request**
 - Include a concise error message describing the failure
-

3. In-Memory Persistence

- Store validated company records in memory only
 - Do not use a database
 - Design the persistence layer so that a database can be introduced later with minimal changes
 - Internal storage structures must not be exposed outside the application layer
-

4. Company Access and Interaction Capabilities

Implement HTTP-based functionality that allows consumers to interact with company data stored in memory.

The API must support:

- Creation of new company records with full validation
- Retrieval of all stored companies
- Retrieval of a single company by a unique identifier
- Retrieval using filtering and search criteria, including:
 - Partial or full company name
 - Website domain
- Relevance-based searching, with results ordered by relevance

- Graceful handling of empty-result scenarios

Behavioral Expectations

- Use appropriate HTTP methods and status codes for all operations
- Return **404 Not Found** when a requested resource does not exist
- Return **400 Bad Request** for invalid input
- Return **200 OK** or **201 Created** for successful operations
- Use DTOs for all external contracts
- Do not expose internal data models or in-memory structures

Design Expectations

- The API surface must follow RESTful principles
 - Route and query design are part of the evaluation
 - Business logic must not reside in controller layers
 - The solution must be extensible to support additional operations such as updates, deletion, pagination, or sorting
-

5. Code Structure and Design Quality

Organize the solution to ensure clarity, maintainability, and extensibility.

Expected qualities:

- Clear separation of concerns
 - Thoughtful use of services and abstractions
 - Minimal coupling between layers
 - Clean, readable, and well-structured code
-

6. Frontend Application (Angular)

Create a simple frontend application using **Angular**.

The frontend must:

- Provide a form to submit:
 - Company Name
 - Website URL
- Display validation success or error messages
- Retrieve and display stored company data using the API

Styling and state management should remain minimal. Functional correctness and integration quality are the primary focus.

7. Engineering Practices

The solution must demonstrate professional engineering standards, including:

- Consistent and correct use of HTTP status codes
 - Clear validation logic and user-friendly error messages
 - Use of modern C#/.NET features (e.g., async/await, records, data annotations)
 - Logging for key operations and error cases
 - Unit tests for core business logic and API behavior
-

Running the Application

Backend

Run the backend using:

```
dotnet run
```

The API will be available at:

- <http://localhost:5000>
- <https://localhost:7000>

Frontend

Run the frontend using the standard Angular development server.

Use the frontend, Postman, or curl to exercise the API.

Submission Instructions

Submit the solution using one of the following methods:

- A GitHub repository link
- A hosted environment
- A zip file containing:
 - Backend source files
 - Frontend project files
 - A README with setup and execution instructions

Ensure the submission includes all files required to run both the backend and frontend.

Review Expectations

Candidates should be prepared to discuss:

- API design decisions and trade-offs
- Validation and relevance strategy
- Code structure and abstraction choices
- Extensibility considerations